

Database Management System Lab (CS 29006)

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

School of Computer Engineering



Strictly for internal circulation (within KIIT) and reference only. Not for outside circulation without permission

1 Credit

Course Committee

Lab Contents



2

Sr #	Major and Detailed Coverage Area	Lab#
1	DQL – Part 2 ☐ Grouping ☐ Character Function ☐ Conversion Function ☐ Mathematical Function ☐ Date Function	4

Data Grouping



3

Grouping data is the process of combining columns with duplicate values in a logical order. For example, a database might contain information about employees; many employees live in different cities, but some employees live in the same city. One might want to execute a query that shows employee information for each particular city and then group employee information by city and creating a summarized report.

Example - perhaps one want to figure the average salary paid to employees according to each city. One can do this by using the aggregate function AVG on the SALARY column, and by using the **GROUP BY** clause to group the output by city.

Grouping data is accomplished through the use of the **GROUP BY** clause of a **SELECT** statement (query).

GROUP BY Clause



4

The **GROUP BY** clause is used in collaboration with the **SELECT** statement to arrange identical data into groups. This clause follows the **WHERE** clause in a **SELECT** statement and precedes the **ORDER BY** clause.

The position of the GROUP BY clause in a query is as follows:

Ordering #	Clause
1	SELECT
2	FROM
3	WHERE
4	GROUP BY
5	ORDER BY

Syntax:

[Table 2] depicts optional

```
SELECT COLUMN1, COLUMN2 FROM TABLE1[, TABLE2] WHERE CONDITIONS  
GROUP BY COLUMN1, COLUMN2 ORDER BY COLUMN1, COLUMN2
```

GROUP BY Clause cont...



5

The **GROUP BY** clause is normally a much more CPU-intensive operation, and if the rows are constrained, unnecessary data that would later be discarded. So intentionally reduce the data set with the **WHERE** clause so to perform grouping only on the rows we need.

One can use the **ORDER BY** statement, but normally the relational database management system (RDBMS) also orders the results by the column ordering in the **GROUP BY** clause, which is discussed more in depth later. So unless one need to order the values in a different pattern than the **GROUP BY** clause, the **ORDER BY** clause is redundant. However, sometimes it is provided because you are using aggregate functions in the **SELECT** statement that are not in the **GROUP BY** clause or because your particular RDBMS functions slightly differently from the standard.

Grouping Selected Data



6

Grouping data is simple. The selected columns (the column list following the SELECT keyword in a query) are the columns you can reference in the GROUP BY clause. **If a column is not in the SELECT statement, you cannot use it in the GROUP BY clause.** How can you group data on a report if the data is not displayed?

If the column name has been qualified, the qualified name must go into the GROUP BY clause.

Example – using SUM function

Table - Sales(OrderID, OrderDate, OrderPrice, OrderQuantity, CustomerName)

Problem Statement - Retrieve a list with unique customers from Sales table, and at the same time to get the total amount each customer has spent in the store.

Solution - SELECT CustomerName, SUM(OrderPrice) FROM Sales GROUP BY CustomerName;

Grouping Selected Data cont...



7

Example – using COUNT function

Table - Customer(ID, FirstName, LastName, City, Country, Phone)

Problem Statement - List the number of customers in each country.

Solution - SELECT COUNT(ID), Country FROM Customer GROUP BY Country;

Example – using MIN function

Table - Agent(ID, Name, City, Phone, Commission)

Problem Statement - List the agent with minimum commission for each city.

Solution - SELECT City, MIN(Commission) FROM Agent GROUP BY City;

Example – using MAX function

Table - Agent(ID, Name, City, Phone, Commission)

Problem Statement - List the agent with maximum commission for each city.

Solution - SELECT City, MAX(Commission) FROM Agent GROUP BY City;

Class Work



8

JOB (ID, NAME, TITLE, SALARY, BONUS)

- ☐ List the minimum and maximum salary for each title.
- ☐ List the number of employees in each title
- ☐ List the total and average salary for each title



Representing Column Names with Numbers

Like the ORDER BY clause, one can order the GROUP BY clause by using an integer to represent the column name.

Example:-

```
SELECT City, MAX(Commission) FROM Agent GROUP BY 1;
```

Group By and Order By



9

GROUP BY clause works the same as the ORDER BY clause in that both sort data. Specifically, you use the ORDER BY clause to sort data from a query. The GROUP BY clause also sorts data from a query to properly group the data.

However, there are some differences and disadvantages of using GROUP BY instead of ORDER BY for sorting operations:

- ❑ All non-aggregate columns selected must be listed in the GROUP BY clause.
Example: *SELECT LAST_NAME, FIRST_NAME, CITY FROM EMPLOYEE GROUP BY LAST_NAME; ? what's wrong?*

Correct ? *SELECT LAST_NAME, FIRST_NAME, CITY FROM EMPLOYEE GROUP BY LAST_NAME, FIRST_NAME, CITY ;*

- ❑ The GROUP BY clause is generally not necessary unless you're using aggregate functions.

Example: *SELECT CITY, LAST_NAME
FROM EMPLOYEE
GROUP BY CITY, LAST_NAME
ORDER BY 2,1;*

Having Clause



10

The HAVING clause, when used in conjunction with the GROUP BY clause in a SELECT statement, tells GROUP BY which groups to include in the output. **HAVING is to GROUP BY as WHERE is to SELECT.** In other words, the WHERE clause places conditions on the selected columns, and the HAVING clause places conditions on groups created by the GROUP BY clause. Therefore, when HAVING clause is used, effectively including or excluding is happening, as the case might be, whole groups of data from the query results.

Ordering #	Clause
1	SELECT
2	FROM
3	WHERE
4	GROUP BY
5	HAVING
6	ORDER BY

Having Clause cont...



11

The following is the syntax of the SELECT statement, including the HAVING clause:

```
SELECT COLUMN1, COLUMN2  
FROM TABLE1, TABLE2  
WHERE CONDITIONS  
GROUP BY COLUMN1, COLUMN2  
HAVING CONDITIONS  
ORDER BY COLUMN1, COLUMN2;
```

Example:

- ❑ `SELECT CustomerName, SUM(OrderPrice) FROM Sales GROUP BY CustomerName HAVING SUM(OrderPrice) > 4000;`
- ❑ `SELECT COUNT(ID), Country FROM Customer GROUP BY Country HAVING COUNT(ID) > 20 ;`

Character Function



12

Character functions are functions that represent strings in SQL in formats different from the way they are stored in the table. Character functions is used mainly to compare, join, search, and extract a segment of a string or a value in a column.

Concatenation

To concatenate JOHN and SON to produce JOHNSON. The code looks like :
SELECT 'JOHN' || 'SON' FROM DUAL;

syntax is COLUMN_NAME || [' ' ||] COLUMN_NAME [COLUMN_NAME]

Example - SELECT CITY || ', ' || STATE FROM EMPLOYEE;

SELECT LAST_NAME + ', ' + FIRST_NAME AS NAME FROM EMPLOYEE;

Translation

TRANSLATE function replaces a sequence of characters in a string with another set of characters. However, it replaces a single character at a time. Syntax is ***TRANSLATE(string1, string_to_replace, replacement_string)***

Example - TRANSLATE('1tech23', '123', '456') and **Result:** '4tech56'

Character Function cont...



13

REPLACE Function

The **REPLACE** function replaces every occurrence of a character or string with another specified character or string. The use of this function is similar to the TRANSLATE function except only one specific character or string is replaced within another string. The syntax is

REPLACE('VALUE', 'VALUE', [NULL] 'VALUE')

Example - SELECT REPLACE(FIRST_NAME, 'T', 'B') FROM EMPLOYEE;
SELECT CITY, REPLACE(CITY, 'I', 'Z') FROM EMPLOYEE;

UPPER Function

It converts lowercase letters to uppercase letters for a specific string.

Example - SELECT UPPER(CITY) FROM EMPLOYEE;

LOWER Function

It converts uppercase letters to lowercase letters for a specific string.

Example - SELECT LOWER(CITY) FROM EMPLOYEE;

Character Function cont...



14

SUBSTR Function

return a portion of char, beginning at character position, substring length characters long . The syntax is

SUBSTR(COLUMN NAME, STARTING POSITION, LENGTH)

Example - SELECT SUBSTR('ABCDEFGH',3,4) "Substring" FROM DUAL;
SELECT EMP_ID, SUBSTR(EMP_ID,1, 3) FROM EMPLOYEE;

INSTR Function

The INSTR functions search string for substring. The function returns an integer indicating the position of the character in string that is the first character of this occurrence. Syntax is INSTR(COLUMN NAME, SUB STRING, [START POSITION [, OCCURRENCE]]);

Example –

SELECT INSTR('CORPORATE FLOOR','OR', 3, 2) "Instring" FROM DUAL;
SELECT INSTR(STATE,'I',1,1) FROM EMPLOYEE; or SELECT INSTR(CITY, 'E')
from emp;

Character Function cont...



15

LTRIM Function

Clip the part of a string. This function and SUBSTRING are in the same family. LTRIM trims characters from the left of a string. The syntax is LTRIM(CCHARACTER STRING [, STRING])

Example - SELECT LTRIM(FIRST_NAME, 'LES') FROM CUSTOMER WHERE FIRST_NAME ='LESLIE';

RTRIM Function

Like LTRIM, the RTRIM function trims characters, but this time from the right of a string.

Example –

SELECT RTRIM(FIRST_NAME, 'ON') FROM EMPLOYEE WHERE FIRST_NAME = 'BRANDON';

SELECT INSTR(STATE,'I',1,1) FROM EMPLOYEE;

Character Function cont...



16

DECODE Function

DECODE function has the functionality of an IF-THEN-ELSE statement.

```
SELECT supplier_name, DECODE(supplier_id, 10000, 'IBM', 10001, 'Microsoft', 10002, 'Hewlett Packard', 'Gateway') result FROM suppliers;
```

The above DECODE statement is equivalent to the following IF-THEN-ELSE statement:

```
IF supplier_id = 10000 THEN
    result := 'IBM';
ELSIF supplier_id = 10001 THEN
    result := 'Microsoft';
ELSIF supplier_id = 10002 THEN
    result := 'Hewlett Packard';
ELSE
    result := 'Gateway';
END IF;
```

Character Function cont...



17

LENGTH Function

The LENGTH function is a common one that finds the length of a string, number, date.

Example - `SELECT DESC, LENGTH(DESC) FROM PRODUCT;`

IFNULL Function

The IFNULL function returns data from one expression if another expression is NULL. You can use IFNULL with most data types; however, the value and the substitute must be the same data type. The syntax is `IFNULL(VALUE, SUBSTITUTION)`

Example - `SELECT PAGER, IFNULL(PAGER,9999999999) FROM EMPLOYEE;`

LPAD Function

LPAD (left pad) is used to add characters or spaces to the left of a string.

Example - `SELECT LPAD(DESC, 30, '.') PRODUCT FROM PRODUCT;`

Character Function cont...



18

RPAD Function

RPAD (left pad) is used to add characters or spaces to the right of a string.
Example - `SELECT RPAD(DESC, 30, '.') PRODUCT FROM PRODUCT;`

Conversion Function

- ☐ Character to Numeric
 - ☐ `SELECT ID, TO_NUMBER(ID) FROM EMPLOYEE;`
- ☐ Numeric to character
 - ☐ `SELECT ID, TO_CHAR(COST) FROM PRODUCT;`
- ☐ Character to Date
 - ☐ `SELECT TO_DATE(DOB) FROM EMPLOYEE;`
- ☐ Date to Character
 - ☐ `SELECT TO_CHAR(SYSDATE, 'Month DD, YYYY HH24:MI:SS') FROM DUAL;`

Mathematical Function



19

Mathematical functions are standard across implementations. Mathematical functions enable you to manipulate numeric values in a database according to mathematical rules. The most commonly used are:

The most common functions include the following:

- ☐ Absolute value (ABS)
- ☐ Power (POWER)
- ☐ Rounding (ROUND)
- ☐ Ceiling and floor values (CEIL(ING), FLOOR)
- ☐ Square root (SQRT)
- ☐ Exponential values (EXP)
- ☐ Sign values (SIGN)
- ☐ SIN, COS, TAN

Further study:

https://docs.oracle.com/cd/E12032_01/doc/epm.921/html_fr_user/frameset.htm?/cd/E12032_01/doc/epm.921/html_fr_user/fr_studio-13-02.html

Date Function



20

The most common functions include the following:

- ❑ **SYSDATE** - Returns the current date and time set for the operating system on which the database resides. E.g. `SELECT SYSDATE FROM DUAL;`
- ❑ **ADD_MONTHS** - Add a month to a date. e.g. `SELECT ADD_MONTHS (SYSDATE, 2) FROM DUAL;`
- ❑ **NEXT_DAY** - Returns the next day of the week as specified. E.g. `SELECT NEXT_DAY(SYSDATE, 'FRI') FROM DUAL;` Options are SUN, MON, TUE, WED, THU, FRI, and SAT.
- ❑ **MONTHS_BETWEEN** - Returns the number of months between two given dates. E.g. `SELECT MONTHS_BETWEEN(SYSDATE+365, SYSDATE-365) FROM DUAL;`
- ❑ **Year** extraction from DATE column - `SELECT EXTRACT(YEAR FROM created_date) FROM PRODUCT;`
- ❑ **Month** extraction from DATE column - `SELECT EXTRACT(MONTH FROM created_date) FROM PRODUCT;`
- ❑ **Day** extraction from DATE column - `SELECT EXTRACT(DAY FROM created_date) FROM PRODUCT;`



Thank You

End of Lab 4

Experiment



22

Reference Table

- ❑ **PRODUCT** with the attributes:
ID, NAME, DESC, STATUS, CREATED_DATE, CREATED_BY, LAST_MODIFIED_DATE, LAST_MODIFIED_BY, COST, INJECTED_DATE
 - ❑ **JOB** table with the attributes:
ID, NAME, TITLE, SALARY, BONUS , INJECTED_DATE
1. Find the maximum cost, minimum cost by each Product Creator.
 2. Find the sum of cost, average cost by each Last Modifier.
 3. Find the number of products created by Indiana D or Ohio D
 4. Find the number of products updated by Michigan R or Illinois M.
 5. Display the sum of all salary.
 6. Compute the average salary value across all non-null bonus values
 7. Find the average, highest and lowest cost for the products.
 8. Display the total number of jobs for null bonus values.
 9. Display the last modified and created date **year, month value** for each product.

Experiment cont...



23

10. Display the JOB name and title followed by a message based on salary. If the salary is $> 1,00,000$ display "High". If the salary is between 50,000 and 100,000 then display "Medium" else display 'Low'.
11. Find the products created in the month of 'May'
12. Find the products created in current year
13. Find 2 to the power 10
14. Find the ceil and floor value for each job salary
15. Find the average bonus for all jobs
16. Count the distinct job title
17. Display the product name, description, created date with the format '20 June 1988'
18. Display the product name, description in the ascending order of the length of the product description.
19. Display the product name, description in the lower and upper case.
20. Display the Concatenation of product name, description, created date and last updated date for active products.